# Proyecto de robótica

Nombre: _____ Fecha: _____

## Qué vas a aprender

- Aplicar las fases del proyecto tecnológico a un robot
- Planificar el trabajo en equipo con roles, tareas y calendario
- Construir y programar un robot móvil que responda a sensores
- Probar, documentar y presentar el proyecto

## Para pensar

**¿Por qué los ingenieros construyen muchos prototipos que fallan antes del robot definitivo?**

## Explicación

### 1.1 Qué es un proyecto de robótica

Un **proyecto** reúne todo lo aprendido para resolver un problema real con un robot. No se trata solo de programar: hay que **diseñar**, **construir**, **probar**, **organizarse** y **comunicar**.

Ejemplos de retos adecuados para 3º de ESO:

- Un **robot siguelíneas** que recorra un circuito lo más rápido posible.
- Un **robot que esquiva obstáculos** y explora una habitación.
- Una **barrera de aparcamiento** automática con contador de plazas.
- Un **invernadero inteligente** que riega y ventila solo.
- Un **brazo robótico** con servos controlado desde una app o una micro:bit.

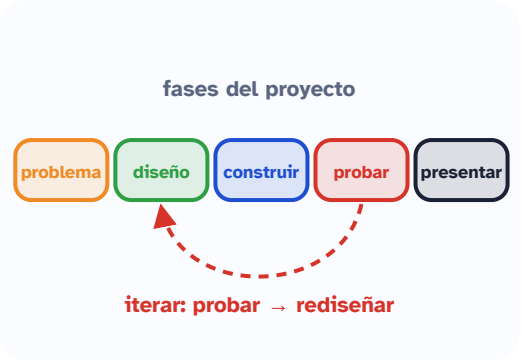
En un proyecto se ponen en juego los contenidos de todo el curso: algoritmos y variables para el programa, sensores y actuadores para el montaje, estructuras y mecanismos para el chasis y herramientas digitales para organizarse y documentar. Por eso es la mejor forma de comprobar lo que se ha aprendido.




### 1.2 Las fases del proyecto

1. **Identificar el problema** y fijar los **requisitos**: qué debe hacer el robot, con qué limitaciones (tamaño, presupuesto, tiempo).
2. **Buscar información** e ideas: soluciones existentes, sensores posibles.
3. **Diseñar**: bocetos, lista de materiales, esquema eléctrico y algoritmo.
4. **Planificar**: tareas, responsables y calendario.
5. **Construir** el prototipo y **programarlo** por partes.
6. **Probar y mejorar** (iterar).
7. **Evaluar, documentar y presentar**.

No es un camino recto: las pruebas suelen obligar a volver al diseño.




## 1.3 Planificar: calendario y diagrama de Gantt

Un proyecto sin calendario casi siempre se queda a medias. Antes de empezar se hace una lista de **tareas** (diseñar el chasis, montar los motores, programar los sensores, escribir la memoria...), se estima cuántas sesiones necesita cada una y se ordenan según sus **dependencias**: no se puede probar el siguelíneas antes de montar los sensores. Después se reparten entre las personas del equipo.

Una herramienta muy útil es el **diagrama de Gantt**: una tabla en la que cada fila es una tarea y cada columna una semana o una sesión; una barra coloreada indica cuándo se hace cada tarea. De un vistazo se ve qué tareas se hacen a la vez, cuáles van con retraso y cuál es la fecha de entrega. Se puede hacer en una hoja de cálculo compartida. Conviene dejar siempre un margen de una o dos sesiones para imprevistos: piezas que no llegan, un motor que se rompe o un fallo difícil de encontrar. Revisar el calendario cada semana permite reaccionar a tiempo y, si hace falta, recortar alguna función menos importante.

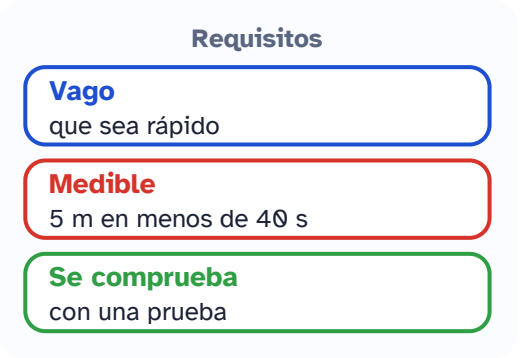



## 2.1 Requisitos claros y medibles

Un buen requisito se puede **comprobar**. «Que el robot sea rápido» es ambiguo; «que recorra el circuito de 5 m en menos de 40 s» es medible.

| Requisito | Cómo se comprueba |
|---|---|
| sigue una línea negra de 2 cm de ancho | completa 3 vueltas sin salirse |
| se detiene a menos de 10 cm de un obstáculo | prueba con una caja, medir con regla |
| cabe en una caja de 20 × 20 cm | medir |
| funciona 15 minutos con una batería | cronometrar |
| presupuesto inferior a 40 € | lista de materiales con precios |

Los requisitos se escriben antes de construir y sirven al final para evaluar el proyecto.




## 2.2 Diseñar: bocetos, esquemas y lista de materiales

Con los requisitos claros, el equipo propone varias ideas y elige la mejor con una **tabla de decisión**: se puntúa cada idea según criterios como coste, dificultad, eficacia y tiempo. La idea elegida se concreta en **bocetos** a mano alzada del robot, vistos desde arriba y de lado, con las medidas principales, y después en planos más precisos o en un modelo 3D si se va a imprimir alguna pieza.

El **esquema eléctrico** indica qué pin de la placa se conecta a cada sensor y actuador; se puede dibujar con un programa como Fritzing o Tinkercad Circuits y permite simular el circuito antes de montarlo. También se escribe el **algoritmo** en pseudocódigo o con un diagrama de flujo. Por último, la **lista de materiales** recoge cada componente, la cantidad, el precio y dónde conseguirlo: así se

calcula el **presupuesto** y se comprueba que cabe en el límite fijado. Reutilizar piezas de otros proyectos abarata el robot y genera menos residuos.



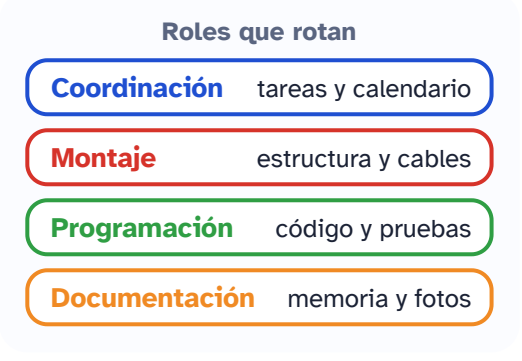

## 2.3 Trabajo en equipo

Los proyectos se hacen en grupos de 3 o 4 personas con **roles** que rotan:

- **Coordinación**: organiza las tareas y vigila el calendario.
- **Diseño y montaje**: estructura, ruedas, cableado.
- **Programación**: algoritmo y código.
- **Documentación**: diario del proyecto, fotos, memoria.

Todos aprenden de todo: los roles indican quién es responsable, no quién trabaja. Herramientas útiles: un **tablero de tareas** (por hacer, en curso, hecho), un calendario con hitos y un **diario** donde anotar decisiones, problemas y soluciones cada sesión. Los desacuerdos se resuelven con datos: probar las dos ideas y quedarse con la que funcione mejor.

Para que el equipo funcione conviene firmar un pequeño acuerdo: cómo se toman las decisiones, qué se hace si alguien falta y cómo se resuelven los desacuerdos. Una reunión breve al principio de cada sesión (qué hice, qué haré, qué me bloquea) mantiene a todo el grupo informado.

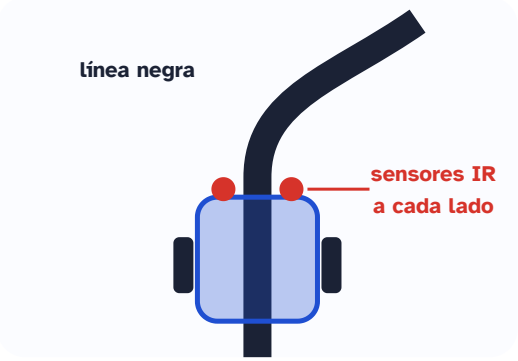


### 3.1 Ejemplo: robot siguelíneas

Un **robot siguelíneas** lleva dos sensores de **infrarrojos** mirando al suelo, a ambos lados de la línea negra. El blanco refleja la luz infrarroja y el negro la absorbe.

| Sensor izquierdo | Sensor derecho | Acción |
|---|---|---|
| blanco | blanco | avanzar recto |
| negro | blanco | girar a la izquierda |
| blanco | negro | girar a la derecha |
| negro | negro | parar (cruce o meta) |

Para girar, se para o se frena la rueda del lado hacia el que se gira. Es un sistema de **lazo cerrado**: mide, corrige y vuelve a medir muchas veces por segundo.

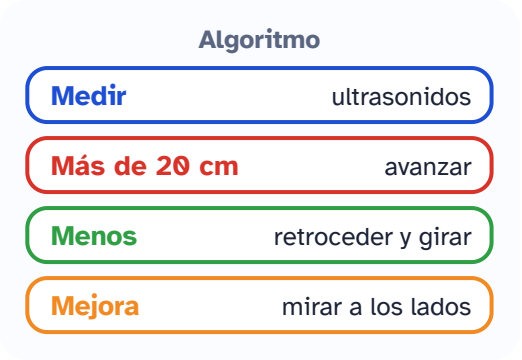



### 3.2 Ejemplo: robot que esquiva obstáculos

Otro proyecto clásico es un robot que avanza por una habitación sin chocar. Lleva un sensor de ultrasonidos en la parte delantera, a menudo montado sobre un servo para poder «mirar» a los lados, dos motores con un driver y una batería. El algoritmo básico es: medir la distancia; si es mayor que 20 cm, avanzar; si es menor, parar, retroceder un poco y girar. Una versión mejorada gira el servo a izquierda y derecha, mide la distancia en cada lado y elige el lado con más espacio libre.

Al probarlo aparecen problemas reales: el sensor no detecta las patas finas de una silla, el robot gira siempre hacia el mismo lado y se queda atrapado en una esquina, o la batería se agota y los motores van más lentos. Cada problema se resuelve con una mejora concreta: añadir sensores de infrarrojos a los lados, girar

un ángulo aleatorio, o medir la tensión de la batería. Así, iteración tras iteración, el robot se vuelve más fiable.




### 3.3 Otro tipo de proyecto: un videojuego

No todos los proyectos de programación necesitan un robot. Un **videojuego** en Scratch o MakeCode Arcade sigue las mismas fases: definir la idea y las reglas (quién es el protagonista, cuál es el objetivo, cómo se gana y se pierde), diseñar los personajes y escenarios, programar por partes y probarlo con otras personas. Un juego de persecución, por ejemplo, empieza con un personaje que se mueve con las flechas; después se añade un perseguidor que lo sigue, un contador de puntos con una variable, un cronómetro y, por último, niveles con dificultad creciente.

Las pruebas con jugadores reales son muy valiosas: detectan si el juego es demasiado difícil, si las instrucciones se entienden o si hay fallos (un personaje que atraviesa paredes). También se puede combinar: un mando hecho con una micro:bit que controla un juego en el ordenador, o un juego que reacciona a sensores reales como la luz o la inclinación.

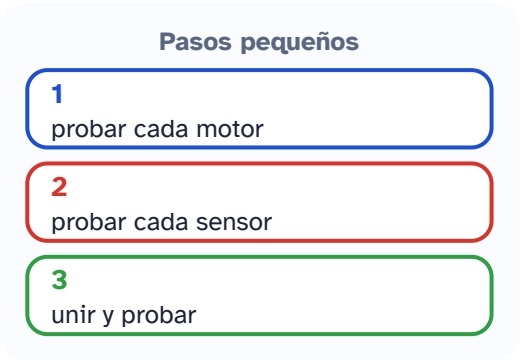



### 3.4 Construir y programar por partes

El error más común es montarlo todo, escribir todo el programa y probarlo al final: si no funciona, no sabes dónde está el fallo. Es mejor avanzar en **pasos pequeños** comprobando cada uno:

1. Probar que **cada motor** gira en el sentido correcto.
2. Leer **cada sensor** y mostrar sus valores (calibrar negro y blanco).
3. Programar **avanzar**, **girar** y **parar** como funciones separadas.
4. Unirlo todo en el bucle principal.
5. Ajustar velocidades y umbrales en el circuito real.

Guarda **versiones** del programa (v1, v2…) para poder volver atrás si un cambio empeora el resultado.

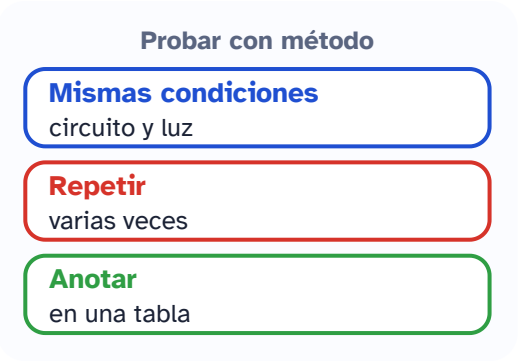



## 4.1 Pruebas y mejora

Las pruebas deben ser **sistemáticas**: mismas condiciones, varias repeticiones y anotar los resultados en una tabla.

| Versión | Cambio | Tiempo (s) | Salidas de pista |
|---|---|---|---|
| v1 | velocidad 50 % | 52 | 0 |
| v2 | velocidad 80 % | 35 | 3 |
| v3 | 80 % en recta, 50 % en curva | 38 | 0 |

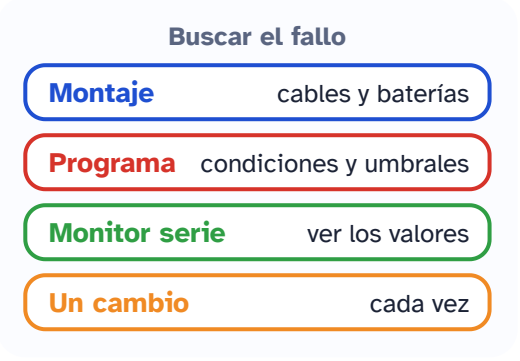
La tabla muestra una situación típica: ir más rápido provoca errores, y la solución es un compromiso. Cambia una sola variable cada vez para saber qué produce cada efecto. Si un problema se repite, anótalo en el diario con su causa y su solución.




## 4.2 Depurar: encontrar y corregir fallos

Cuando el robot no hace lo esperado, hay que **depurar**: encontrar la causa del fallo y corregirla. Lo primero es separar si el problema es de **montaje** (un cable suelto, una batería descargada, un motor conectado al revés) o de **programa** (una condición mal escrita, un umbral incorrecto, un bucle que nunca termina). Probar cada parte por separado con pequeños programas de prueba ayuda a localizarlo.

Algunas técnicas útiles: mostrar los valores de los sensores en el **monitor serie** o en la pantalla de la micro:bit para ver qué está leyendo el robot; encender un LED cuando el programa entra en una parte concreta del código; hacer una **traza** a mano siguiendo las instrucciones; o explicar el programa línea a línea a un compañero (muchas veces el fallo aparece al explicarlo). Hay que cambiar una sola cosa cada vez y comprobar el efecto. Y apuntar en el diario del proyecto cada fallo encontrado y cómo se solucionó: será útil para la memoria y para la próxima vez.



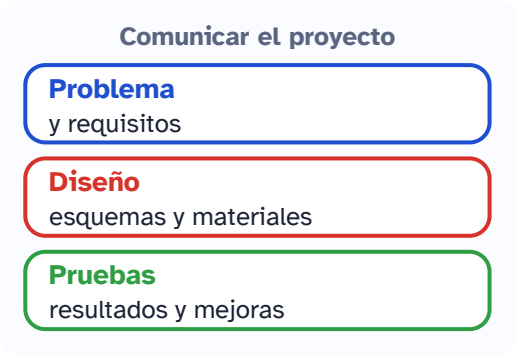

## 5.1 Documentar y presentar

Un proyecto no termina hasta que se **comunica**. La **memoria** del proyecto incluye:

- Problema y requisitos.

- Diseño: bocetos, esquema eléctrico, lista de materiales y presupuesto.
- Algoritmo (diagrama de flujo) y código comentado.
- Pruebas realizadas y mejoras.
- Conclusiones: qué se cumplió, qué no y qué se haría distinto.

En la **presentación** (unos 5 minutos) se hace una demostración en vivo, se explican las decisiones y se responde a preguntas. Un vídeo corto del robot funcionando es un buen complemento por si algo falla ese día.




## 5.2 Presentar el proyecto

La presentación final suele durar entre cinco y diez minutos y combina explicación y **demostración**. Una buena estructura es: el problema que se quería resolver, los requisitos, el diseño (con fotos y esquemas), una demostración en vivo del robot, los resultados de las pruebas con datos y lo que se mejoraría con más tiempo. Todos los miembros del equipo deben participar y conocer el proyecto completo, no solo su parte.

Hay que preparar la demostración con cuidado: llevar baterías cargadas, probarla antes en el mismo lugar y tener un **vídeo** de reserva por si algo falla en directo, porque los robots a veces se comportan distinto con otra luz u otro suelo. Las diapositivas deben tener pocas palabras y muchas imágenes. Después, el público hace preguntas y el equipo responde con honestidad, incluyendo los fallos: explicar qué no funcionó y por qué también demuestra que se ha aprendido. Publicar el código y las instrucciones con una licencia libre permite que otros reutilicen el proyecto.



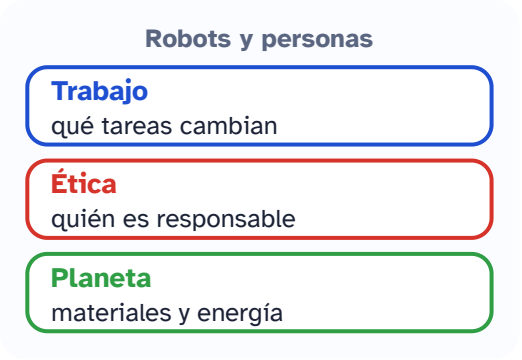

## 5.3 Robótica, sociedad y sostenibilidad

Los robots ya trabajan en fábricas, almacenes, hospitales, el campo o el espacio. Plantean preguntas que también forman parte del proyecto:

- ¿Qué **trabajos** cambian o desaparecen y cuáles nuevos aparecen?
- ¿Quién es **responsable** si un robot autónomo se equivoca?
- ¿Cómo se **reciclan** los componentes y las baterías?

En tu proyecto puedes aplicar criterios **sostenibles**: reutilizar materiales (cartón, tapones, motores de juguetes viejos), usar baterías recargables y desmontar al final para que los componentes sirvan a otros grupos.

Un buen cierre del proyecto es una reflexión escrita del equipo: qué impacto tendría el robot si se fabricara de verdad, quién saldría beneficiado, a quién podría perjudicar y cómo se podría reciclar al final de su vida útil.



## Del reto al robot

| | |
|---|---|
| **Requisitos** | qué debe hacer, medible y comprobable |
| **Diseño** | bocetos · materiales · esquema · algoritmo |
| **Equipo** | roles · tablero de tareas · diario |
| **Construir y probar** | por partes · una variable cada vez · versiones |
| **Comunicar** | memoria · demostración · conclusiones |

## Ejercicios

1. **Ordena las fases: probar, requisitos, construir, presentar, diseñar.**

2. **Reescribe como requisito medible: «el robot debe ser ligero».**

3. **¿Qué cuatro roles puede tener un equipo de proyecto?**

4. **Explica por qué el negro y el blanco se distinguen con un sensor de infrarrojos.**

5. **Si el sensor derecho ve negro y el izquierdo blanco, ¿hacia dónde gira el robot?**

6. **Haz la lista de materiales de un robot que esquiva obstáculos.**

7. **¿Por qué es útil guardar versiones del programa?**

8. **Diseña una tabla para registrar las pruebas de tu robot.**

9. **Nombra dos medidas para que tu proyecto sea más sostenible.**

10. **Planifica un proyecto de barrera de aparcamiento: requisitos, componentes, algoritmo y calendario de 4 semanas.**

11. **¿Qué es un diagrama de Gantt?**

12. **¿Qué incluye la lista de materiales?**

13. **¿Qué es depurar?**

14. **Nombra dos herramientas para dibujar y simular el esquema eléctrico.**

15. **¿Por qué hay que llevar un vídeo de reserva a la presentación?**

16. **¿Qué tres preguntas se hacen en la reunión breve de cada sesión?**

17. **Haz una tabla de decisión con dos ideas para un robot que riega plantas y tres criterios.**

18. **Calcula el presupuesto: placa 22 €, 2 motores a 4 € cada uno, sensor 3 €, driver 5 €, batería 9 €.**

19. **Escribe el algoritmo del robot que esquiva obstáculos con un umbral de 20 cm.**

20. **Clasifica los fallos como de montaje o de programa: motor al revés, umbral incorrecto, cable suelto, bucle sin fin.**

21. **¿Qué son las dependencias entre tareas? Pon un ejemplo.**

22. **Diseña las reglas de un juego de persecución en Scratch.**

23. **¿Por qué es importante cambiar una sola cosa cada vez al depurar?**

24. **Escribe la estructura de una presentación de 7 minutos de tu proyecto.**

25. **Propón tres mejoras para el robot que esquiva obstáculos.**

26. **Planifica completo un proyecto de maceta inteligente: requisitos medibles, materiales y presupuesto, algoritmo y calendario de 4 semanas.**

27. **Diseña una tabla de pruebas para comparar dos versiones del siguelíneas y explica cómo decidirás cuál es mejor.**

28. **Un compañero no participa en el proyecto. ¿Cómo lo resolveríais en el equipo de forma justa?**

29. **Argumenta ventajas e inconvenientes de publicar el código del proyecto con licencia libre.**

30. **Evalúa el impacto social de un robot de reparto en tu barrio: beneficios, riesgos y una propuesta para reducirlos.**

## Practica

1. **Repetir el ciclo diseñar, probar y mejorar se llama...**

   **A)** compilar
   **B)** iterar
   **C)** calibrar
   **D)** programar

2. **Un requisito bien escrito debe ser...**

   **A)** ambiguo
   **B)** secreto
   **C)** medible
   **D)** rápido

3. **La primera versión funcional de un robot para probar ideas es un...**

   **A)** prototipo
   **B)** manual
   **C)** presupuesto
   **D)** examen

4. **Un robot siguelíneas usa sensores de...**

   **A)** temperatura
   **B)** sonido
   **C)** infrarrojos
   **D)** presión

5. **Si el sensor izquierdo ve negro y el derecho blanco, el robot debe...**

   **A)** girar a la derecha
   **B)** girar a la izquierda
   **C)** parar
   **D)** acelerar

6. **Al probar, conviene cambiar...**

   **A)** una sola variable cada vez
   **B)** todo a la vez
   **C)** nada nunca
   **D)** medir la temperatura

7. **La persona que organiza tareas y calendario tiene el rol de...**

**A)** programación
**B)** coordinación
**C)** montaje
**D)** documentación

8. **El documento final que recoge todo el proyecto es la...**

**A)** protoboard
**B)** traza
**C)** memoria
**D)** factura

9. **Guardar v1, v2, v3 del programa sirve para...**

**A)** poder volver atrás
**B)** ocupar memoria
**C)** nada
**D)** decorar

10. **Un criterio sostenible en un proyecto es...**

**A)** usar pilas desechables
**B)** reutilizar materiales
**C)** tirar los componentes
**D)** gastar más pilas

11. **La tabla con tareas y barras de tiempo es un diagrama de...**

**A)** flujo
**B)** Gantt
**C)** Venn
**D)** barras de errores

12. **La lista de materiales sirve para calcular...**

**A)** la velocidad
**B)** el presupuesto
**C)** la distancia
**D)** la nota

13. **Para elegir entre varias ideas se usa una...**

**A)** tabla de decisión
**B)** traza
**C)** protoboard

**D)** variable

14. **Un robot que esquiva obstáculos suele usar un sensor de…**

**A)** humedad
**B)** ultrasonidos
**C)** temperatura
**D)** gas

15. **Encontrar la causa de un fallo y corregirla es…**

**A)** depurar
**B)** compilar
**C)** soldar
**D)** imprimir

16. **Para ver qué lee un sensor mientras pruebas se usa…**

**A)** el monitor serie
**B)** un relé
**C)** un puente H
**D)** la memoria

17. **Un cable suelto es un fallo de…**

**A)** programa
**B)** montaje
**C)** diseño gráfico
**D)** presupuesto

18. **En la presentación conviene llevar…**

**A)** un vídeo de reserva
**B)** el robot sin probar
**C)** solo texto
**D)** nada

19. **Un juego de persecución en Scratch cuenta los puntos con…**

**A)** una variable
**B)** un relé
**C)** un sensor de luz
**D)** un servo

20. **Dejar sesiones libres en el calendario sirve para…**

**A)** imprevistos

**B)** descansar siempre

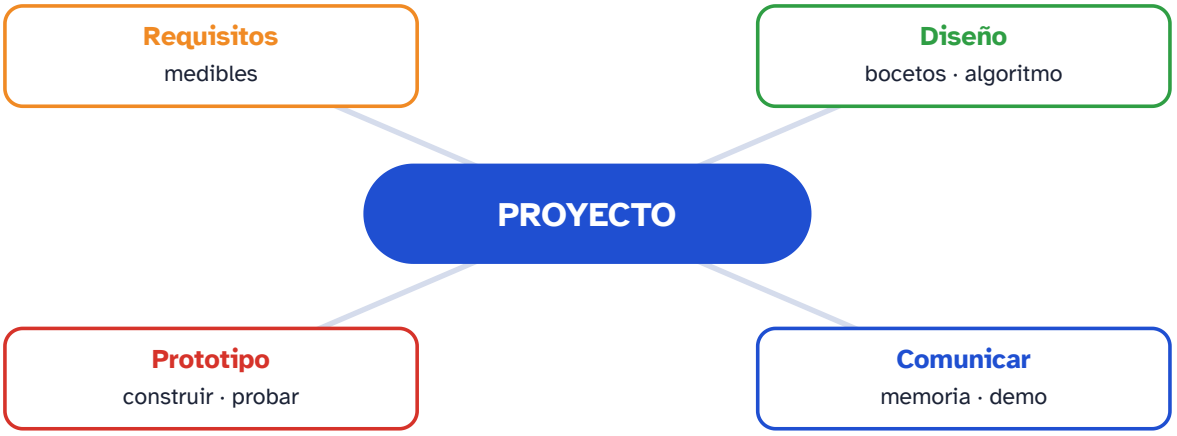
**C)** nada

**D)** gastar más

## Explícalo con tus palabras

Incluye: Las fases de un proyecto de robótica; Cómo escribir un requisito medible; Cómo decide un robot siguelíneas hacia dónde girar; Por qué conviene construir y probar por partes.

## Resumen




## Glosario

| | |
|---|---|
| **Proyecto** | Trabajo organizado para resolver un problema real. |
| **Requisito** | Lo que debe cumplir la solución, de forma medible. |
| **Prototipo** | Primera versión funcional para probar ideas. |
| **Iterar** | Repetir diseñar, probar y mejorar. |
| **Diagrama de Gantt** | Calendario de tareas con barras. |
| **Dependencia** | Tarea que necesita que termine otra. |
| **Tabla de decisión** | Puntuación de ideas según criterios. |
| **Boceto** | Dibujo rápido a mano alzada. |
| **Esquema eléctrico** | Dibujo de las conexiones del circuito. |
| **Lista de materiales** | Componentes, cantidades y precios. |
| **Presupuesto** | Coste total previsto. |
| **Rol** | Responsabilidad de cada persona del equipo. |
| **Depurar** | Encontrar y corregir fallos. |
| **Prueba sistemática** | Prueba repetida en las mismas condiciones. |
| **Memoria** | Documento que recoge todo el proyecto. |
| **Licencia libre** | Permiso para reutilizar y compartir. |

## Tarjetas de repaso

Recórtalas: pregunta delante, respuesta detrás.

**Requisito**

Lo que debe cumplir el robot, de forma medible.

**Prototipo**

Primera versión funcional para probar ideas.

**Iterar**

Repetir diseñar → probar → mejorar.

**Roles**

Responsabilidades en el equipo: coordinación, montaje, programación, documentación.

**Diario del proyecto**

Registro de decisiones, problemas y soluciones.

**Siguelíneas**

Robot que sigue una línea con sensores infrarrojos.

**Prueba sistemática**

Mismas condiciones, varias repeticiones, una variable cada vez.

**Memoria**

Documento final con todo el proyecto.

**Diagrama de Gantt**

Tareas en filas y barras de tiempo.

**Lista de materiales**

Componentes, cantidad y precio.

**Depurar**

Encontrar y corregir fallos.

**Demostración**

Mostrar el robot funcionando en vivo.

## Soluciones

1. **B)** Iteración.
2. **C)** Se puede comprobar.
3. **A)** Se mejora tras las pruebas.
4. **C)** El negro absorbe la luz IR.
5. **B)** Volver sobre la línea.
6. **A)** Para saber qué causa cada efecto.
7. **B)** Los roles rotan.
8. **C)** Problema, diseño, pruebas, conclusiones.
9. **A)** Control de versiones.
10. **B)** Y baterías recargables.
11. **B)** Planificación.
12. **B)** Precio de cada pieza.
13. **A)** Puntuar criterios.
14. **B)** Mide la distancia.
15. **A)** Depuración.
16. **A)** Muestra los valores.
17. **B)** Parte física.
18. **A)** Por si falla en directo.
19. **A)** Contador.
20. **A)** Margen.

## Soluciones de los ejercicios

1. Requisitos, diseñar, construir, probar, presentar (con iteraciones).
2. Por ejemplo: «el robot debe pesar menos de 500 g».
3. Coordinación, diseño y montaje, programación y documentación.
4. El blanco refleja la luz infrarroja y el negro la absorbe, así que el sensor recibe más o menos luz.
5. Hacia la derecha.
6. Placa (Arduino o micro:bit), 2 motores con ruedas, rueda loca, controlador de motores, sensor de ultrasonidos, batería, chasis y cables.
7. Para volver a una versión que funcionaba si un cambio empeora el robot.
8. Columnas: versión, cambio realizado, resultado medido (tiempo, errores), observaciones.
9. Reutilizar materiales y usar baterías recargables (también desmontar y reutilizar componentes).
10. Respuesta abierta; debe incluir sensor de presencia, servo para la barrera, contador de plazas y fases repartidas en semanas.
11. Una tabla con las tareas en filas y el tiempo en columnas, donde barras indican cuándo se hace cada tarea.
12. Cada componente, la cantidad, el precio y dónde conseguirlo.
13. Encontrar la causa de un fallo y corregirla.
14. Fritzing y Tinkercad Circuits.
15. Por si la demostración en directo falla por la luz, el suelo o las baterías.
16. Qué hice, qué haré y qué me bloquea.
17. Criterios: coste, dificultad y eficacia, puntuados de 1 a 5; se suma y gana la idea con más puntos.
18. 22 + 8 + 3 + 5 + 9 = 47 €.
19. Repetir siempre: medir distancia; si d > 20 avanzar; si no, parar, retroceder 0,5 s y girar 0,5 s.
20. Montaje: motor al revés y cable suelto. Programa: umbral incorrecto y bucle sin fin.
21. Tareas que no pueden empezar hasta que termine otra: no se puede probar el sensor sin haberlo montado.
22. El jugador se mueve con flechas; un enemigo lo persigue; suma un punto por cada moneda; pierde una vida al ser tocado; gana con 10 puntos o pierde con 0 vidas.
23. Para saber exactamente qué cambio arregla (o empeora) el comportamiento.
24. Problema (1 min), requisitos y diseño (2 min), demostración (2 min), resultados de pruebas (1 min), mejoras y preguntas (1 min).
25. Mirar a los lados con un servo, añadir infrarrojos laterales y avisar cuando la batería está baja.
26. Requisitos: mantener la humedad entre 40 y 60 %, avisar con LED. Materiales: placa, sonda, relé, bomba pequeña, LED, batería (unos 35 €). Algoritmo con histéresis. Calendario: diseño, montaje, programa y pruebas, memoria.

27. Columnas: versión, intento, tiempo, salidas de pista. 10 intentos por versión en el mismo circuito y con la misma luz. Gana la que tenga menos salidas y, a igualdad, menor tiempo medio.
28. Hablar con él para entender la causa, revisar el reparto y los roles, darle una tarea concreta con fecha y, si sigue sin participar, comunicarlo al profesor según el acuerdo del equipo.
29. Ventajas: otros pueden aprender, reutilizarlo y mejorarlo; recibir sugerencias. Inconvenientes: otros pueden copiarlo sin aportar; hay que cuidar la documentación y no publicar datos personales.
30. Beneficios: menos coches y entregas rápidas. Riesgos: ocupación de aceras, pérdida de empleos, privacidad por las cámaras. Propuesta: rutas y velocidades limitadas, formación de los trabajadores y normas sobre las imágenes.